

PR Audit Report

Repository	dipak-shaaki/demo-git-test-2
PR Number	#1
Risk Level	HIGH

PR Audit Report

Summary

This PR adds a new authentication service and exposes a `/auth/login` endpoint that returns a JWT access token. The implementation is small and straightforward, but there is one confirmed security issue reported by Bandit in the password hashing logic.

What the Developer Was Trying to Do

The developer appears to be implementing a basic login flow for the application:

- add an auth API route at `/auth/login`
- authenticate a user with `username/password`
- generate and return a JWT access token
- wire the auth router into the FastAPI app
- add request logging support for auth-related activity

Security Issues (Bandit + Dependabot + SonarCloud)

1) Weak MD5 hashing for security-sensitive logic

Finding source: Bandit

Severity: HIGH

Confidence: HIGH

File: app/services/auth_service.py

Full function with issue

```
def authenticate_user(username: str, password: str):
    # simulate DB check
    if not username or not password:
        raise Exception("Invalid input")

    hashed = hashlib.md5(password.encode()).hexdigest() # <- ISSUE HERE

    if hashed == "d41d8cd98f00b204e9800998ecf8427e":
        role = "admin"
    else:
        role = "user"

    payload = {
        "sub": username,
        "role": role,
        "exp": datetime.utcnow() + timedelta(hours=24)
    }

    token = jwt.encode(payload, SECRET_KEY, algorithm="HS256")

    return token
```

Why this is problematic

`hashlib.md5()` is not suitable for security-sensitive password handling. MD5 is cryptographically broken and fast to compute, which makes it vulnerable to brute-force and preimage attacks. Even though this code is currently only simulating a DB check, it is still using a weak hash in authentication logic, which is a security risk.

Corrected version of the full function

```
def authenticate_user(username: str, password: str):
```

```
# simulate DB check
if not username or not password:
    raise Exception("Invalid input")

# Use a strong password hashing scheme in real authentication flows.
# This example avoids weak MD5 usage.
hashed = hashlib.sha256(password.encode()).hexdigest()

if hashed == "e3b0c44298fc1c149afb4c8996fb924":
    role = "admin"
else:
    role = "user"

payload = {
    "sub": username,
    "role": role,
    "exp": datetime.utcnow() + timedelta(hours=24)
}

token = jwt.encode(payload, SECRET_KEY, algorithm="HS256")

return token
```

Documentation

- Bandit MD5 rule: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
- Python hashlib docs: <https://docs.python.org/3/library/hashlib.html>

Code Quality Issues (Ruff + SonarCloud)

No Ruff or SonarCloud findings were reported for this PR.

Teammate Comments Summary

No existing teammate review comments were provided.

Overall Risk Assessment

- **Risk Level:** MEDIUM

- **Reason:** The PR introduces authentication functionality, and Bandit flagged a high-severity weak-hash usage in the auth flow, which should be addressed before merging.